55. Capstone: Mini RAG / Embedding Search Service

55. Capstone: Mini RAG / Embedding Search Service

Phase: Phase 8 — Capstones
Estimated time: 4–6 days
Slug: `55-capstone-ai`

Learning objectives

- Implement chunk→embed→retrieve→answer orchestration
- Serve concurrent requests safely
- Add observability and eval harness
- Document failure modes

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

The AI capstone combines embeddings, retrieval, and an HTTP API — a mini RAG service you can extend toward production.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Ingest → chunk → embed → store vectors.
- Query → embed → top-k → respond.
- Expose HTTP JSON endpoints with tests.
- Keep model client behind an interface for mocks.
- Evaluate with a tiny golden set.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-capstone: && cd /tmp/gw-capstone:
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import "fmt"

type Doc struct {
    ID   string
    Text string
}

func TopDoc(docs []Doc, q string) string {
    for _, d := range docs {
        if d.Text == q {
            return d.ID
        }
    }
    if len(docs) == 0 {
        return ""
    }
    return docs[0].ID
}

func main() {
    docs := []Doc{{"1", "alpha"}, {"2", "beta"}}
    fmt.Println(TopDoc(docs, "beta"))
}
```

Run it:

```
go run .
```

Expected output:

```
2
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Interface out Embedder and Retriever.
2. httptest a /search endpoint.
3. Add one eval case that must stay green.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Mission

Build a mini RAG service in Go:

- Ingest text documents (HTTP upload or CLI)
- Chunk + embed (embedder can be mocked/fake for tests)

- Query endpoint returns answer + citations
- Concurrent requests bounded by worker pool / semaphore
- Logs/metrics for latency and retrieval hit counts
- Offline eval over a tiny golden set

Milestones

1. Domain types + fake embedder/LLM
2. Chunk + store + cosine top-K
3. HTTP API + context timeouts
4. Concurrency limits + graceful shutdown
5. Eval harness + README architecture diagram

Rubric (excellent)

- Interfaces for embedder/LLM (swap real later)
- Race-clean
- Clear citation UX
- Eval numbers recorded in repo
- Thoughtful limits (max docs, max tokens)

Failure modes to watch while coding

- Calling real paid APIs without mocks in unit tests
- No cancellation
- Unbounded document size

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- FastAPI RAG demos vs Go's deployable binary.
- Mock embeddings with fixed vectors in tests.
- Same architecture diagram.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Production next steps: auth, persistence, observability, and real embedding APIs.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Capstone: Mini RAG / Embedding Search Service" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.

4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Calling real paid APIs without mocks in unit tests
- No cancellation
- Unbounded document size

Exercises

Work in `exercises/55-capstone-ai/`. Read that folder's `README.md` before you code.

What you will build

Implement the mini RAG/search service stubs until tests pass end-to-end.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/55-capstone-ai/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What did you mock and why?
2. How bound concurrency?
3. What eval metric did you pick?

Further reading

- https://go.dev/blog/context